// CYBER SECURITY SERIES — SCRIPTING //

Bash Scripting

Cyber Security-র জন্য সম্পূর্ণ গাইড

Basics থেকে Security Automation পর্যন্ত

Bash কী সেটা না জানলেও সমস্যা নেই — একদম শুরু থেকে variables, loops, functions শিখে নিজের Port Scanner, Log Analyzer, Network Monitor, Vulnerability Checker এবং Security Automation Scripts তৈরি করতে শেখো।

```
root@kali:~# ./network_scanner.sh 192.168.1.0/24
[+] Scanning network ...
[✓] 192.168.1.1 - ALIVE (Gateway)
[✓] 192.168.1.5 - ALIVE (Open: 22,80,443)
[!] 192.168.1.10 - ALIVE (Open: 23 - Telnet!)
```

📝 Bash Basics | 🔄 Loops & Conditions | ⚙ Functions | 📁 File Operations | 🌐 Network Tools | 📡 Port Scanner | 📊 Log Analyzer | 🔒 Security Scripts

🤖 Automation 📋 Cheat Sheet

// বিষয়সূচি //

BASH SCRIPTING FOR CYBER SECURITY — COMPLETE GUIDE

01 **Bash পরিচিতি ও প্রথম Script**
Shell কী, Bash কেন, shebang, chmod, execution
BASICS

02 **Variables, Input ও String Operations**
Variable declaration, user input, string manipulation, arrays
VARS

03 **Conditions ও Comparisons**
if/elif/else, test operators, case statement, logical operators
LOGIC

04 **Loops — for, while, until**
IP range loop, wordlist iteration, break/continue
LOOPS

05 **Functions ও Script Structure**
Function তৈরি, arguments, return values, modular scripting
FUNC

06 **File Operations ও Text Processing**
grep, awk, sed, cut, sort, uniq — Security-তে অপরিহার্য
FILES

07 **Network Tools ও Scanning**
ping, nc, nmap, curl, wget — Bash দিয়ে network operation
NET

08 🔥 **Port Scanner Script তৈরি**
নিজের পূর্ণাঙ্গ port scanner — Banner grabbing সহ
PROJECT

09 🔥 **Log Analyzer Script**
auth.log থেকে brute force, suspicious IP detect করা
PROJECT

10 🔥 **Security Automation Scripts**
System hardening checker, vulnerability scanner, backup script
PROJECT

11 **Advanced Bash — Tips ও Best Practices**
Error handling, debugging, getopts, cron, parallel execution
ADVANCED

12

Cheat Sheet — সম্পূর্ণ রেফারেন্স

Bash syntax, Security commands, One-liners — সব এক জায়গায়

CHEAT

CHAPTER 01

🖥 Bash পরিচিতি ও প্রথম Script

Shell কী, Bash কেন শিখবে, script লেখা ও চালানো

Bash কী এবং কেন?

Bash (Bourne Again Shell) হলো Linux-এর সবচেয়ে বেশি ব্যবহৃত command-line interpreter। Cyber Security-তে Bash অপরিহার্য কারণ এটি দিয়ে repetitive tasks automate করা যায়, tools একসাথে combine করা যায় এবং custom security tools তৈরি করা যায়।

✅ কেন Bash শিখব?

- Security tools automate করা
- Log file analyze করা
- Network scan script তৈরি
- CTF challenges solve করা
- OSCP-তে অপরিহার্য

📂 Script File তৈরি করা

- `Extension: .sh`
- `Shebang: #!/bin/bash`
- `chmod +x script.sh`
- `./script.sh`
- `bash script.sh`

🔴 যেকোনো Text Editor ব্যবহার করো

- nano script.sh
- vim script.sh
- VS Code

- gedit script.sh

প্রথম Bash Script

```
#!/bin/bash # ↑ Shebang — এই line বলে কোন interpreter ব্যবহার করতে হবে # hello.sh — প্রথম bash script echo "হ্যালো! আমার প্রথম Bash Script!" echo "আজকের তারিখ: $(date)" echo "আমি কে: $(whoami)" echo "আমি কোথায়: $(pwd)"
```

```
# Script চালানো $ chmod +x hello.sh # Execute permission দাও $ ./hello.sh # চালাও # অথবা সরাসরি $ bash hello.sh
```

গুরুত্বপূর্ণ Special Variables

```
#!/bin/bash echo "Script নাম: $0" echo "প্রথম argument: $1" echo "দ্বিতীয় argument: $2" echo "সব arguments: $@" echo "Argument সংখ্যা: $#" echo "শেষ command-এর exit code: $?" echo "Current process ID: $$" # Argument সহ চালানো: # ./script.sh 192.168.1.1 80 # $1 = 192.168.1.1, $2 = 80
```

Comments ও Best Practices

```
#!/bin/bash ############################################## # Script: security_check.sh # Author: তোমার নাম # Date: 2026-04-05 # Description: Basic security check script ############################################## # Color codes for output RED='\033[0;31m' GREEN='\033[0;32m' YELLOW='\033[1;33m' BLUE='\033[0;34m' NC='\033[0m' # No Color echo -e "${GREEN}[+]${NC} Script শুরু হচ্ছে..." echo -e "${RED}[!]${NC} সতর্কতা!" echo -e "${YELLOW}[*]${NC} তথ্য"
```

// মূল কথা //

- ✅ প্রতিটি script-এ #!/bin/bash দিয়ে শুরু করো
- ✅ chmod +x দিয়ে execute permission দাও

- ✅ Comments লেখো — ভবিষ্যতে কাজে আসবে
- ✅ Color codes ব্যবহার করো — output সুন্দর হয়

CHAPTER 02

📦 Variables, Input ও String Operations

Data store করা, user থেকে input নেওয়া এবং string manipulate করা

Variables তৈরি করা

```
#!/bin/bash # Variable তৈরি — = এর দুই পাশে কোনো space নেই! target_ip="192.168.1.1"
target_port=80 scan_name="My Pentest" # Variable ব্যবহার করা — $ দিয়ে echo "Target:
$target_ip" echo "Port: ${target_port}" # {} দিয়ে স্পষ্ট করা # Command output variable-এ রাখা
current_user=$(whoami) current_date=$(date +%Y-%m-%d) ip_address=$(hostname -I | awk
'{print $1}') echo "User: $current_user | Date: $current_date | IP: $ip_address" #
Readonly variable (constant) readonly VERSION="1.0" # Variable unset করা unset target_port
```

User Input নেওয়া

```
#!/bin/bash # Basic input read -p "Target IP দাও: " target echo "Scanning: $target" #
Silent input (password) read -sp "Password: " password echo # newline # Timeout সহ input
read -t 10 -p "10 সেকেন্ডে উত্তর দাও: " answer # Default value read -p "Port (default 80): "
port port=${port:-80} # যদি খালি থাকে তাহলে 80 # Command line argument validation if [ "$#"
-lt 1 ]; then echo "Usage: $0 <target_ip>" exit 1 fi target="$1"
```

String Operations — Security-তে গুরুত্বপূর্ণ

```
#!/bin/bash url="https://target.com/admin/login.php" ip="192.168.1.100" # String length
echo "Length: ${#url}" # Substring extraction echo "${url:0:5}" # https echo "${url: -8}"
# শেষ 8 chars # String replace echo "${url/https/http}" # প্রথমটি replace echo "${url//o/0}"
# সব replace # Contains check if [[ "$url" == *"admin"* ]]; then echo "[!] Admin path
found!" fi # IP থেকে octets বের করা IFS='.' read -ra octets <<< "$ip" echo "Network:
```

```
${octets[0]}.${octets[1]}.${octets[2]}.0/24" # Arrays ports=(21 22 23 80 443 8080 3306)
echo "First port: ${ports[0]}" echo "All ports: ${ports[@]}" echo "Count: ${#ports[@]}"
```

CHAPTER 03

🔀 Conditions ও Comparisons

if/elif/else, test operators, case statement — সিদ্ধান্ত নেওয়ার কৌশল

if / elif / else

```
#!/bin/bash port=22 if [ "$port" -eq 22 ]; then echo "SSH port — check for weak auth" elif [ "$port" -eq 80 ]; then echo "HTTP — check for XSS, SQLi" elif [ "$port" -eq 443 ]; then echo "HTTPS — check SSL config" elif [ "$port" -eq 23 ]; then echo "TELNET — CRITICAL! Unencrypted!" else echo "Unknown service on port $port" fi
```

Comparison Operators

Operator	Numbers-এর জন্য	Strings-এর জন্য
সমান	`-eq`	`==`
সমান নয়	`-ne`	`!=`
বড়	`-gt`	`>` (with [[]])
ছোট	`-lt`	`<` (with [[]])
বড় বা সমান	`-ge`	
ছোট বা সমান	`-le`	

File ও Directory Checks

```
#!/bin/bash logfile="/var/log/auth.log" target="$1" # File exist করে কি না if [ ! -f "$logfile" ]; then echo "[-] Log file পাওয়া যাচ্ছে না!" exit 1 fi # Directory exist check if [ ! -d "/output" ]; then mkdir -p /output fi # File operators # -f → regular file # -d → directory # -e → exists (যেকোনো) # -r → readable # -w → writable # -x → executable # -s →
```

```
non-empty file # -z → empty string # -n → non-empty string # Argument check if [ -z "$target" ]; then echo "Error: Target দাও!" exit 1 fi # Logical operators if [ -f "$logfile" ] && [ -r "$logfile" ]; then echo "Log file readable!" fi
```

Case Statement

```
#!/bin/bash service="$1" case "$service" in "ssh"|"22") echo "SSH: Check weak passwords, key-based auth" ;; "http"|"80") echo "HTTP: Check XSS, SQLi, directory listing" ;; "ftp"|"21") echo "FTP: Check anonymous login" ;; "smb"|"445") echo "SMB: Check EternalBlue, null session" ;; *) echo "Unknown: $service" ;; esac
```

CHAPTER 04

🔄 Loops — for, while, until

IP range loop, wordlist iterate, repetitive task automate করা

for Loop

```
#!/bin/bash # ১. List-এ loop for port in 21 22 23 80 443 3306 8080; do echo "Checking port: $port" done # ২. Range-এ loop for i in {1..254}; do echo "192.168.1.$i" done # ৩. C-style loop for (( i=1; i<=100; i++ )); do echo "Port $i" done # ৪. File/wordlist iterate করা while read -r ip; do echo "Scanning: $ip" ping -c 1 -W 1 "$ip" &>/dev/null && echo "ALIVE: $ip" done < targets.txt # ৫. Glob pattern for file in /var/log/*.log; do echo "Analyzing: $file" done
```

while ও until Loop

```
#!/bin/bash # while loop — condition সত্য হওয়া পর্যন্ত count=1 while [ "$count" -le 10 ]; do echo "Attempt: $count" (( count++ )) done # Ping monitor — host down না হওয়া পর্যন্ত host="8.8.8.8" while ping -c 1 "$host" &>/dev/null; do echo "$(date): $host is UP" sleep 60 done echo "ALERT: $host is DOWN!" # until loop — condition মিথ্যা হওয়া পর্যন্ত count=0 until [ "$count" -ge 5 ]; do echo "Count: $count" (( count++ )) done # break এবং continue for port in {1..1000}; do if [ "$port" -eq 500 ]; then break # 500-এ থামো fi if [ "$port" -eq 100 ]; then continue # 100 skip করো fi echo "Port: $port" done
```

CHAPTER 05

⚙ Functions ও Script Structure

Reusable functions তৈরি করা — professional security script-এর ভিত্তি

Function তৈরি ও ব্যবহার

```
#!/bin/bash # Color variables RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m';
NC='\033[0m' # Basic function function print_banner() { echo
"═══════════════════════════════" echo " Security Scanner v1.0" echo
"═══════════════════════════════" } # Function with arguments function log_result() {
local status="$1" # local = function-এর ভেতরেই সীমাবদ্ধ local message="$2" local
timestamp=$(date +"%H:%M:%S") case "$status" in "success") echo -e "${GREEN}[+]${NC}
[$timestamp] $message" ;; "fail") echo -e "${RED}[-]${NC} [$timestamp] $message" ;;
"info") echo -e "${YELLOW}[*]${NC} [$timestamp] $message" ;; esac } # Return value সহ
function function is_port_open() { local host="$1" local port="$2" timeout 1 bash -c "echo
>/dev/tcp/$host/$port" 2>/dev/null return $? # 0 = open, 1 = closed } # Function call করা
print_banner log_result "info" "Scanning শুরু হচ্ছে..." if is_port_open "192.168.1.1" 80;
then log_result "success" "Port 80 OPEN!" else log_result "fail" "Port 80 CLOSED" fi
```

Professional Script Structure

```
#!/bin/bash ############################################ # Security Tool Template
############################################ # ── CONSTANTS ──────────────────────────
readonly SCRIPT_NAME=$(basename "$0") readonly VERSION="1.0" readonly
LOG_FILE="/tmp/scan_$(date +%Y%m%d).log" RED='\033[0;31m'; GREEN='\033[0;32m'
YELLOW='\033[1;33m'; BLUE='\033[0;34m'; NC='\033[0m' # ── FUNCTIONS
──────────────────────────── function usage() { echo "Usage: $SCRIPT_NAME [OPTIONS]
<target>" echo " -p PORT Port to scan" echo " -h This help" exit 0 } function
check_dependencies() { local deps=("nmap" "curl" "nc") for dep in "${deps[@]}"; do if !
command -v "$dep" &>/dev/null; then echo -e "${RED}[-]${NC} $dep installed নেই!" exit 1 fi
done echo -e "${GREEN}[+]${NC} সব dependencies OK" } # ── ARGUMENT PARSING
──────────────────── while getopts "p:h" opt; do case "$opt" in p) PORT="$OPTARG" ;; h)
usage ;; *) usage ;; esac done shift $(( OPTIND - 1 )) TARGET="$1" # ── MAIN
──────────────────────────── function main() { check_dependencies echo -e "${BLUE}
```

```
[*]${NC} Starting scan on $TARGET" # Main logic এখানে } # Script-কে directly run করা হলেই
main() call হবে if [[ "${BASH_SOURCE[0]}" == "${0}" ]]; then main fi
```

CHAPTER 06

📂 File Operations ও Text Processing

grep, awk, sed, cut, sort — Security log analysis-এর অপরিহার্য tools

File Read ও Write

```
#!/bin/bash # File লেখা echo "192.168.1.1" > targets.txt # overwrite echo "192.168.1.2" >> targets.txt # append # Multi-line লেখা (heredoc) cat > report.txt << EOF ═══ Security Scan Report ═══ Date: $(date) Target: $TARGET EOF # File পড়া line by line while IFS= read -r line; do echo "Processing: $line" done < targets.txt # File-এর তথ্য echo "Lines: $(wc -l < file.txt)" echo "Words: $(wc -w < file.txt)" echo "Size: $(du -sh file.txt)"
```

grep — Pattern খোঁজা

```
# Basic grep grep "Failed password" /var/log/auth.log grep -i "error" /var/log/syslog # case-insensitive grep -v "#" config.txt # নেই এমন line grep -c "Failed" /var/log/auth.log # count grep -n "root" /etc/passwd # line number সহ grep -r "password" /var/www/ # recursive # Extended grep (regex) grep -E "[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}" file.txt grep -E "Failed|Invalid|error" /var/log/auth.log grep -oE "[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+" log.txt # শুধু IP extract # Security use cases grep "Failed password" /var/log/auth.log | awk '{print $11}' | sort | uniq -c | sort -rn # → কোন IP সবচেয়ে বেশি fail করেছে
```

awk — Data Processing

```
# awk — column/field প্রক্রিয়া করা # $1=first field, $2=second field, NF=last field # /etc/passwd থেকে username বের করা awk -F: '{print $1}' /etc/passwd # 5th column print করা awk '{print $5}' /var/log/auth.log # Condition সহ awk '$3 > 100 {print $1, $3}' data.txt # Auth log থেকে failed IP extract awk '/Failed password/ {print $11}' /var/log/auth.log # Sum calculation awk '{sum += $1} END {print "Total:", sum}' numbers.txt
```

sed — Stream Editor

```
# sed — text replace, delete, insert # Replace করা sed 's/old/new/g' file.txt # সব replace
sed -i 's/password/****/g' config.txt # file-এ directly # Line delete করা sed '/^#/d'
config.txt # comment lines বাদ sed '/^$/d' file.txt # empty lines বাদ # নির্দিষ্ট line print করা
sed -n '5,10p' file.txt # line 5-10 sed -n '/ERROR/p' log.txt # ERROR আছে এমন line #
Practical: IP list থেকে comments ও blanks বাদ sed '/^#/d; /^$/d' targets.txt
```

Useful One-liners

```
# sort ও uniq sort -u file.txt # Unique, sorted sort -rn file.txt # Numeric, descending
uniq -c file.txt | sort -rn | head -10 # Most frequent # cut — delimiter দিয়ে column বের
করা cut -d: -f1 /etc/passwd # : delimiter, 1st field cut -d',' -f2 data.csv # CSV 2nd
column # tr — character translate echo "hello" | tr a-z A-Z # uppercase echo "a:b:c" | tr
':' '\n' # : → newline # Piping — tools combine করা cat /var/log/auth.log | grep "Failed"
| awk '{print $11}' | sort | uniq -c | sort -rn | head -20
```

CHAPTER 07

🌐 Network Tools ও Scanning

ping, nc, curl, nmap — Bash দিয়ে network operation এবং reconnaissance

ping — Host Discovery

```
#!/bin/bash # Single ping test ping -c 1 -W 1 192.168.1.1 &>/dev/null && echo "UP" || echo "DOWN" # Subnet sweep for i in {1..254}; do ping -c 1 -W 1 192.168.1.$i &>/dev/null && \ echo "[+] ALIVE: 192.168.1.$i" & # & = background-এ চালাও done wait # সব background job শেষ হওয়া পর্যন্ত অপেক্ষা
```

nc (Netcat) — Swiss Army Knife

```
# Port check nc -zv 192.168.1.1 80 2>&1 | grep "succeeded" # Bash দিয়ে port check (nc ছাড়া) timeout 1 bash -c "echo >/dev/tcp/192.168.1.1/80" 2>/dev/null && echo "OPEN" # Banner grabbing echo "" | nc -w 2 192.168.1.1 21 # FTP banner echo "" | nc -w 2 192.168.1.1 22 # SSH banner echo -e "HEAD / HTTP/1.0\r\n\r\n" | nc -w 2 192.168.1.1 80 # Listener (authorized testing only) nc -lvnp 4444 # Listen on port 4444
```

curl ও wget — HTTP Requests

```
# Basic curl curl https://target.com curl -s -o /dev/null -w "%{http_code}" https://target.com # status code curl -I https://target.com # headers only curl -L https://target.com # follow redirects # POST request curl -X POST -d "user=admin&pass=test" https://target.com/login # Custom headers curl -H "Authorization: Bearer TOKEN" https://api.target.com/data # Security headers check function check_headers() { local url="$1" local headers=$(curl -s -I "$url") echo "═══ Security Headers: $url ═══" for header in "Strict-Transport-Security" "Content-Security-Policy" "X-Frame-Options" "X-Content-Type-Options"; do if echo "$headers" | grep -qi "$header"; then echo -e " ✓ $header" else echo -e " ✗ MISSING: $header" fi done } check_headers "https://example.com"
```

nmap Integration

```
#!/bin/bash function nmap_scan() { local target="$1" local output_dir="./results" mkdir -p
"$output_dir" echo "[*] Quick scan শুরু ..." nmap -sV --top-ports 1000 -T4 "$target" \ -oN
"$output_dir/${target}_quick.txt" \ -oX "$output_dir/${target}_quick.xml" # Open ports
extract করা echo "[+] Open ports:" grep "/open/" "$output_dir/${target}_quick.txt" | awk
'{print $1, $3}' # Vulnerability scan echo "[*] Vulnerability scan চালাচ্ছি ..." nmap --script
vuln "$target" -oN "$output_dir/${target}_vuln.txt" } nmap_scan "$1"
```

CHAPTER 08 — PROJECT

🔥 Port Scanner Script তৈরি

নিজের পূর্ণাঙ্গ port scanner — banner grabbing, service detection সহ

08

> ⚠️ **শুধুমাত্র নিজের নেটওয়ার্কে বা authorized lab-এ ব্যবহার করো**

সম্পূর্ণ Port Scanner Script

```
#!/bin/bash ############################################ # BashScan — Simple Port Scanner # Usage: ./bashscan.sh <target> [start_port] [end_port] ############################################ # Colors RED='\033[0;31m'; GREEN='\033[0;32m' YELLOW='\033[1;33m'; BLUE='\033[0;34m'; NC='\033[0m' # Default values TARGET="${1}" START_PORT="${2:-1}" END_PORT="${3:-1000}" TIMEOUT=1 OPEN_PORTS=() # Service name lookup function get_service() { case "$1" in 21) echo "FTP" ;; 22) echo "SSH" ;; 23) echo "TELNET ⚠️" ;; 25) echo "SMTP" ;; 53) echo "DNS" ;; 80) echo "HTTP" ;; 110) echo "POP3" ;; 143) echo "IMAP" ;; 443) echo "HTTPS" ;; 445) echo "SMB ⚠️" ;; 3306) echo "MySQL" ;; 3389) echo "RDP" ;; 6379) echo "Redis ⚠️" ;; 8080) echo "HTTP-Alt" ;; *) echo "Unknown" ;; esac } # Banner grabbing function grab_banner() { local host="$1" local port="$2" local banner banner=$(timeout 2 bash -c "echo '' | nc -w2 $host $port 2>/dev/null | head -1") echo "${banner//[^[:print:]]/}" # non-printable chars বাদ } # Port scan function function scan_port() { local port="$1" timeout "$TIMEOUT" bash -c "echo >/dev/tcp/$TARGET/$port" 2>/dev/null if [ "$?" -eq 0 ]; then local service=$(get_service "$port") local banner=$(grab_banner "$TARGET" "$port") printf " ${GREEN}[OPEN]${NC} %-6s %-12s %s\n" "$port" "$service" "$banner" OPEN_PORTS+=("$port") fi } # Validation if [ -z "$TARGET" ]; then echo "Usage: $0 <target> [start] [end]" exit 1 fi # Banner echo -e "${BLUE}" echo " ╔═══════════════════╗" echo " ║ BashScan Port Scanner ║" echo " ╚═══════════════════╝" echo -e "${NC}" echo -e " Target : ${GREEN}$TARGET${NC}" echo -e " Range : ${YELLOW}$START_PORT - $END_PORT${NC}" echo -e " Started: $(date)\n" echo " PORT SERVICE BANNER" echo " ──────────────────────────────────" # Main scan loop for (( port=START_PORT; port≤END_PORT; port++ )); do scan_port "$port" & # Background-এ চালাও # Thread limit — একসাথে বেশি হলে system overload if (( port % 100 == 0 )); then wait fi done wait # সব background job শেষ হওয়া পর্যন্ত # Summary echo "" echo "
```

```
━━━
```

CHAPTER 09 — PROJECT

📊 Log Analyzer Script

09

auth.log বিশ্লেষণ করে brute force attack, suspicious IP ও anomaly detect করা

সম্পূর্ণ Log Analyzer Script

```
#!/bin/bash ############################################ # LogHunter — Security Log Analyzer # Usage: ./loghunter.sh [logfile] ############################################
RED='\033[0;31m'; GREEN='\033[0;32m' YELLOW='\033[1;33m'; BLUE='\033[0;34m'; NC='\033[0m'
LOGFILE="${1:-/var/log/auth.log}" THRESHOLD=10 # এই সংখ্যার বেশি fail = suspicious
REPORT="/tmp/security_report_$(date +%Y%m%d_%H%M%S).txt" # Check file if [ ! -f "$LOGFILE"
]; then echo -e "${RED}[-]${NC} Log file পাওয়া যায়নি: $LOGFILE" exit 1 fi function
print_section() { echo -e "\n${BLUE}══ $1 ══${NC}" } function analyze_auth_log() {
print_section "Failed SSH Login Attempts" echo " Top attacking IPs:" grep "Failed
password" "$LOGFILE" 2>/dev/null | \ awk '{print $11}' | \ sort | uniq -c | sort -rn |
head -10 | \ while read count ip; do if [ "$count" -ge "$THRESHOLD" ]; then echo -e "
${RED}[!]${NC} $ip → $count attempts (SUSPICIOUS)" else echo -e " ${YELLOW}[*]${NC} $ip →
$count attempts" fi done print_section "Invalid Users" echo " Top non-existent usernames
tried:" grep "Invalid user" "$LOGFILE" 2>/dev/null | \ awk '{print $8}' | \ sort | uniq -c
| sort -rn | head -10 | \ while read count user; do echo -e " ${YELLOW}[*]${NC} '$user' →
$count times" done print_section "Successful Logins" echo " Recent successful SSH logins:"
grep "Accepted password\|Accepted publickey" "$LOGFILE" 2>/dev/null | \ tail -10 | awk
'{print $1,$2,$3,"User:",$9,"From:",$11}' | \ while read line; do echo -e " ${GREEN}
[+]${NC} $line" done print_section "Root Login Attempts" root_fails=$(grep "Failed
password for root" "$LOGFILE" 2>/dev/null | wc -l) if [ "$root_fails" -gt 0 ]; then echo -
e " ${RED}[!!!] Root login failed $root_fails times!" else echo -e " ${GREEN}[+]${NC} No
root login failures" fi print_section "Summary Statistics" echo -e " Total failed logins :
$(grep -c 'Failed' "$LOGFILE" 2>/dev/null)" echo -e " Unique attacking IPs: $(grep 'Failed
password' "$LOGFILE" 2>/dev/null | awk '{print $11}' | sort -u | wc -l)" echo -e " Log
file size : $(du -sh "$LOGFILE" | cut -f1)" echo -e " Log date range : $(head -1
"$LOGFILE" | cut -d' ' -f1-3) → $(tail -1 "$LOGFILE" | cut -d' ' -f1-3)" } # Header echo -
e "${GREEN}" echo " ╔═══════════════════════════╗" echo " ║ LogHunter - Log Analyzer ║"
echo " ╚═══════════════════════════╝" echo -e "${NC}" echo -e " File:
```

```
${YELLOW}$LOGFILE${NC}" echo -e " Time: $(date)\n" analyze_auth_log echo -e "\n${GREEN}
[+]${NC} Analysis complete!"
```

CHAPTER 10 — PROJECT

🤖 Security Automation Scripts

10

System hardening checker, web security tester, backup automation

System Hardening Checker

```
#!/bin/bash # system_hardening_check.sh — নিজের Linux system check করো PASS=0; FAIL=0;
WARN=0 GREEN='\033[0;32m'; RED='\033[0;31m'; YELLOW='\033[1;33m'; NC='\033[0m' function
check_pass() { echo -e " ${GREEN}[PASS]${NC} $1"; (( PASS++ )); } function check_fail() {
echo -e " ${RED}[FAIL]${NC} $1"; (( FAIL++ )); } function check_warn() { echo -e "
${YELLOW}[WARN]${NC} $1"; (( WARN++ )); } echo "═══ System Security Check ═══" echo
"Target: $(hostname) | $(date)" echo "" # ── SSH SECURITY ── echo "── SSH Configuration
──" sshd_config="/etc/ssh/sshd_config" if grep -q "PermitRootLogin no" "$sshd_config"
2>/dev/null; then check_pass "Root SSH login disabled" else check_fail "Root SSH login
ENABLED — disable করো!" fi if grep -q "PasswordAuthentication no" "$sshd_config"
2>/dev/null; then check_pass "Password auth disabled (key-only)" else check_warn "Password
auth enabled — key-based-এ switch করো" fi # ── FIREWALL ── echo ""; echo "── Firewall ──"
if systemctl is-active --quiet ufw 2>/dev/null || \ systemctl is-active --quiet firewalld
2>/dev/null; then check_pass "Firewall active" else check_fail "Firewall inactive — ufw
enable করো!" fi # ── UPDATES ── echo ""; echo "── System Updates ──" updates=$(apt list --
upgradable 2>/dev/null | grep -c upgradable) if [ "$updates" -le 5 ]; then check_pass
"System up to date ($updates pending)" else check_warn "$updates updates pending — apt
upgrade করো" fi # ── DANGEROUS SERVICES ── echo ""; echo "── Dangerous Services ──" for
service in telnet ftp rsh rexec; do if systemctl is-active --quiet "$service" 2>/dev/null;
then check_fail "$service is RUNNING — disable করো!" else check_pass "$service not
running" fi done # ── EMPTY PASSWORDS ── echo ""; echo "── User Security ──"
empty_pass=$(awk -F: '($2 == "" ) {print $1}' /etc/shadow 2>/dev/null) if [ -z
"$empty_pass" ]; then check_pass "No empty passwords" else check_fail "Empty password
users: $empty_pass" fi # ── SUMMARY ── echo "" echo "═══════════════════════════════" echo
-e " ${GREEN}PASS: $PASS${NC} | ${YELLOW}WARN: $WARN${NC} | ${RED}FAIL: $FAIL${NC}" echo
"═══════════════════════════════"
```

Automated Backup Script

```
#!/bin/bash # secure_backup.sh — গুরুত্বপূর্ণ file backup BACKUP_DIR="/backup/$(date +%Y-%m-%d)" IMPORTANT_DIRS=("/etc" "/home" "/var/www") LOG="/var/log/backup.log" mkdir -p "$BACKUP_DIR" function log() { echo "[$(date '+%H:%M:%S')] $*" | tee -a "$LOG" } log "Backup শুরু হচ্ছে..." for dir in "${IMPORTANT_DIRS[@]}"; do if [ -d "$dir" ]; then name=$(basename "$dir") tar -czf "$BACKUP_DIR/${name}_$(date +%Y%m%d).tar.gz" \ --exclude="*.tmp" "$dir" 2>/dev/null log "[OK] $dir backup complete" fi done # 7 দিনের পুরনো backup মুছে ফেলা find /backup -type d -mtime +7 -exec rm -rf {} + 2>/dev/null log "Backup complete! Location: $BACKUP_DIR"
```

CHAPTER 11

🚀 Advanced Bash — Tips ও Best Practices

Error handling, debugging, parallel execution, cron job — professional level scripting

Error Handling

```
#!/bin/bash # Best practices — script শুরুতেই এগুলো রাখো set -e # error হলে script বন্ধ হয়ে যাবে set -u # undefined variable = error set -o pipefail # pipe-এ error হলে catch করবে set -x # Debug mode (development-এ) # Trap — script বন্ধ হওয়ার আগে cleanup function cleanup() { echo "Cleaning up..." rm -f /tmp/temp_scan_$$ } trap cleanup EXIT SIGINT SIGTERM # Error handling with custom message function die() { echo -e "\033[0;31m[ERROR]${NC} $1" >&2 exit 1 } nmap -sV 192.168.1.1 || die "Nmap scan failed!"
```

Debugging

```
# Debug mode চালানো bash -x script.sh # প্রতিটা command দেখাবে bash -v script.sh # input lines দেখাবে bash -n script.sh # syntax check (run করবে না) # Script-এর ভেতরে debug on/off set -x # debug শুরু # ... debug করার code ... set +x # debug বন্ধ # Variable value check echo "DEBUG: target=$target, port=$port" >&2
```

Parallel Execution — দ্রুত Scanner

```
# ১. Background processes for ip in {1..254}; do ping -c 1 192.168.1.$ip &>/dev/null && echo "ALIVE: 192.168.1.$ip" & done wait # ২. xargs দিয়ে parallel seq 1 254 | xargs -P 50 -I{} ping -c 1 -W 1 192.168.1.{} \ | grep "bytes from" | awk '{print $4}' # ৩. Semaphore — thread limit MAX_JOBS=20 current_jobs=0 for port in {1..1000}; do scan_port "$port" & (( current_jobs++ )) if (( current_jobs ≥ MAX_JOBS )); then wait -n # যেকোনো একটা শেষ হওয়া পর্যন্ত (( current_jobs-- )) fi done wait
```

Cron Job — Automated Scheduling

```
# Cron job edit করা crontab -e # Cron syntax: মিনিট ঘন্টা দিন মাস বার # প্রতি ঘন্টায় log check 0 *
* * * /home/user/loghunter.sh >> /var/log/security.log 2>&1 # প্রতিদিন রাত ২টায় backup 0 2 *
* * /home/user/secure_backup.sh # প্রতি ৫ মিনিটে network monitor */5 * * * *
/home/user/network_monitor.sh # প্রতি সোমবার system check 0 9 * * 1
/home/user/system_hardening_check.sh | mail -s "Security Report" admin@example.com # Cron
job দেখা crontab -l
```

CHAPTER 12

📋 Cheat Sheet — সম্পূর্ণ রেফারেন্স

Bash syntax, Security one-liners, Network commands — সব এক জায়গায়

Bash Syntax Quick Reference

Variables & Strings

`var="value"`	Variable set
`echo "$var"`	Print variable
`var=$(command)`	Command output
`${#var}`	String length
`${var:-default}`	Default value
`${var/old/new}`	Replace
`arr=(a b c)`	Array create
`${arr[@]}`	All array items

Control Flow

`if [ cond ]; then`	Condition
`[ -f file ]`	File exists
`[ -z "$var" ]`	Empty string
`[ "$a" -eq "$b" ]`	Numbers equal
`for i in {1..10}`	Range loop
`while read line`	Read file
`case "$var" in`	Case statement
`function fn() {}`	Function

Security One-liners

```
# ── NETWORK DISCOVERY ── # Live hosts find করা for i in {1..254}; do ping -c1 -W1
192.168.1.$i &>/dev/null && echo "192.168.1.$i"; done # Port check (nc ছাড়া) timeout 1 bash
-c "echo >/dev/tcp/192.168.1.1/22" && echo "Port 22 OPEN" # ── LOG ANALYSIS ── # Top
attacking IPs grep "Failed password" /var/log/auth.log | awk '{print $11}' | sort | uniq -
c | sort -rn | head -20 # Successful logins grep "Accepted" /var/log/auth.log | awk
'{print $9,$11,$3}' | sort -u # Error analysis grep -i "error\|fail\|warn" /var/log/syslog
| awk '{print $5}' | sort | uniq -c | sort -rn # ── FILE SEARCH ── # Sensitive files খোঁজা
find / -name "*.conf" -o -name "*.cfg" 2>/dev/null | xargs grep -l "password" 2>/dev/null
# SUID files খোঁজা find / -perm -u=s -type f 2>/dev/null # World-writable files খোঁজা find /
-perm -o+w -type f 2>/dev/null | grep -v "/proc\|/sys" # ── SYSTEM INFO ── # Listening
ports ss -tulpn | grep LISTEN # Running processes ps aux | grep -v "^root" | awk '{print
$1,$11}' | sort -u # Open connections ss -tn | grep ESTABLISHED | awk '{print $5}' | sort
| uniq -c | sort -rn # ── WEB SECURITY ── # Security headers check curl -sI
https://example.com | grep -i "strict\|content-security\|x-frame\|x-content" # Response
code check while read url; do code=$(curl -s -o /dev/null -w "%{http_code}" "$url") echo
"$code $url" done < urls.txt
```

গুরুত্বপূর্ণ Commands Quick Ref

Text Processing

`grep -E "pattern" file`	Regex search
`awk '{print $1}' file`	First column
`sed 's/old/new/g'`	Replace
`cut -d: -f1 file`	Cut field
`sort | uniq -c`	Count unique
`sort -rn`	Numeric descending
`wc -l file`	Line count
`head -20 / tail -20`	First/last lines

Network Commands

`nc -zv HOST PORT`	Port check
`curl -I URL`	HTTP headers
`curl -s -o /dev/null -w "%{http_code}"`	Status code
`ss -tulpn`	Open ports

`dig domain ANY`	DNS lookup
`whois domain`	Domain info
`tcpdump -i eth0 port 80`	Capture traffic
`nmap -sV --top-ports 1000`	Port scan

```
$ chmod +x security.sh && ./security.sh
```

[+] Bash basics শেখা হলো ✓
[+] Security scripts তৈরি করা শেখা হলো ✓
[+] Automation master হয়ে গেলে ✓

সম্পূর্ণ শিক্ষামূলক উদ্দেশ্যে। শুধুমাত্র authorized environment ও নিজের system-এ ব্যবহার করো।

v1.0 — Bash Scripting for Cyber Security | Bilingual | 12 Chapters + Projects + Cheat Sheet